PROOF OF KEAGS

Posted on 2026.05.12

SUBSCRIBE → EMAIL ATOM RSS

When I was in my first year of professional work, I had an experience that has changed my view on the process of writing software forever.

20yo Keags wrote some C++ code that was crashing. Full-blown segfaults with no real information about what was happening. Every time I thought I had an idea of what was wrong it seemed I'd come up short. I spent two whole work days from 8am to 3am just trying to make sense of what my code was doing.

When I finally figured out what was wrong, it turned out the source of the issue was so ridiculously far from the point where it crashed that it defied any intuition.

What happened? I allocated heap memory using parentheses instead of brackets:

```
int* x = new int(3);
```

instead of

```
int* x = new int[3];
```

array; what I got instead was a one-integer "array" whose value was initialized to "3".

When I understood the cause of the issue, I got so angry, because this was the type of mistake that wasn't a flaw in my actual reasoning, but in the distance between what I thought I was writing and what I was actually writing. This is the kind of thing our tools should help prevent.

It was this episode that set me down the path in search of better ways to write software, and eventually led me to learning Haskell at the beginning of 2017.

I guarantee you every working engineer has at least one story like this.

The Discipline We Don't Use

We treat these stories as the cost of doing business, but we've known how to do better for at least four decades. The body of work — dependent types, refinement types, separation logic, model checking, theorem proving — exists, the foundations are well-understood, and serious software does ship this way.

- CompCert is a verified C compiler used in flight control systems.
- seL4 is a verified microkernel running in defense systems.

The techniques work. They're in production. So why doesn't everyone use them?

When an ordinary engineering team sits down to build a new web service, wallet, compiler, or pipeline, none of these tools come up. Verified software has remained a permanent academic curiosity that occasionally ships in safety-critical contexts and never anywhere else. The natural assumption is that the techniques don't actually work. Except they do. The real reason is that the techniques have been too expensive *for humans*. And that is the assumption AI is invalidating right in front of our eyes here in 2026.

The Limiting Factor

The bottleneck on verified software is not a lack of verification methods. It's the costs of using them. These costs occur along three axes simultaneously, and the field has historically had to compromise on at least one to ship.

1. Writability: writing strong types is tedious.

2. Cultural Literacy: finding people who can write them well is harder.
3. The Performance-Correctness Fork: even if you do, you pay

Writability

Languages with type systems strong enough to express interesting properties are hard for humans to write fluently.

This isn't a minor concern; how onerous a language is to write has been one of the dominant forces shaping which languages get used. Dynamic typing won large swaths of the industry over static typing not because dynamic types are better, but because, at the time, type inference wasn't yet solvable at scale and the types you'd have to write by hand felt verbose and intrusive.

Even when more rigorous tools made code easier to maintain, the daily friction of writing with them kept them off most teams' shortlist. The price of expressing "this list always has at least three elements" through dependent types — like in Rocq, Agda, Lean, or F* — is a discipline most production engineers will not pay during normal feature work.

Cultural Literacy

Even when the techniques work, the economics of staffing a team often don't.

Haskell is the prime example. It is one of the most rigorous, well-designed languages ever built, with wild stories of doing codebase-wide refactors that work on the first try… and yet, it

embedded in it fluently are too rare to staff a normal team. The language demands a literacy the broader software industry never invested in.

The Performance-Correctness Fork

There has always been a budget tradeoff between performance and correctness.

It's not that the laws of computing pit these two goals against one another, but given finite engineering effort, teams must choose between using that effort to make the product faster, and using it to make the product more provably correct.

Optimization and verification have been competing for the same scarce human attention, and this practical tension is the environment in which our software-writing tools evolved. The result is that embedded in the landscape of our tools there is a reasonably discernible fault-line between those designed for speed of execution and those designed for correctness. Verification has always lived in functional / proof-assistant languages which are good for reasoning, but historically either slow or unpredictable when it comes to execution costs.

To bridge this gap and ship on real silicon, verified programs had to be *extracted* to C or another source language with a "battle-tested" runtime story. Running pure functional code

Each axis is a real constraint, and each compromise — write less powerful types, hire more slowly, accept slower runtimes — has been the way the industry accommodated reality. Verification stayed niche while the rest of the field shipped C, Python, and JavaScript. We learned to just live with the bugs, because, let's be real, most bugs are things we *can* live with, even if we'd prefer not to.

This is the equilibrium that's disrupted by AI code workflows.

What AI Changes

AI coding agents change all three constraints in a way where the new gradient points towards verification rather than away from it.

Writability

Writability is a relationship between language and author, but as of 2026, the primary authors of software are no longer human.

In the evolution of programming language design we have seen that the languages that demand less specificity while still retaining the ability to run tend to beat out languages that

strong reluctance to work in dynamic languages for serious projects. Sturdier languages demand more of their authors, not just in terms of clear intent, but in terms of raw structured input. When your ability to ship is in part gated by how many lines of well-formed code you can physically punch into the computer, the increased expense of writing a well-formed line of code adds up considerably.

On top of this, humans in corporate environments have a felt need for a defensible metric of productivity. Lines of working code serve as an important barometer for work getting done. These vanity metrics tilt us away from using verifiable languages because the very nature of verification demands total correctness and makes the notion of intermediate progress somewhat incoherent and therefore immeasurable.

When AI authors the code, that friction disappears. The human is no longer the one paying the per-line attention cost. The human reads, judges, and decides — the writing happens at the speed of inference, and the emotional economics that shaped human language preferences become irrelevant.

What is left, when writability stops mattering, is *legibility*. What gets written, occasionally still has to be read. It has to be reviewable. A human being — me, a teammate, or a regulator —

language optimized for legibility even at the expense of writability is exactly what this regime encourages, and it happens to be the property the powerful languages always had.

Writing in Lean is demanding. *Reading* a well-written Lean program is much easier than reading equivalent C, because the types are a far more effective distillation of intent than the procedure that carries that intent out.

Cultural Literacy

Cultural literacy has been a critical property of a programming language ecosystem because industrial software production has been a team game. To ensure our ability to work as a team, it has been important that languages be accessible to as many people as possible because it allows us to tap into the largest pool of cognitive labor.

Engineering is hard and requires education no matter what, but verified languages have a higher minimum bar of education that has to be met before an engineer can ship code that solves real-world problems. Every engineer on your team has to clear that bar to be productive. Clearing that bar is a cost that has to be

paid by every individual and cannot be easily transferred between them.

types, equational reasoning, referential transparency, function totality — that the industry never developed at scale.

The nature of this training can be thought of as a capital expenditure (CapEx) whereas the cost of fixing quality problems arising from this lack of education can be thought of as an operating expenditure (OpEx). Corporations are rightly reluctant to take on steep CapEx without an assurance of tenure that would allow it to pay off. Individuals similarly don't bother frontloading this education because it isn't a limiting factor to their employment.

Training a model to operate in Lean is a one-time CapEx investment that replicates as fast as you can add inference capacity. There is no intrinsic shelf-life on that training the way there would be for human engineers. An engineer's coding career is often quite short as most promotion tracks push engineers away from direct code authorship. However, the investment the industry refused to make in Haskell, it implicitly makes in every model trained on a corpus that includes Lean and F* and Rocq.

Three things compound with this.

First, the same property that makes powerful languages efficient for human learners — tighter feedback loops, more constraints caught at compile time — makes them extremely efficient for AI

Second, given comparable training material, an AI will operate in Lean *more* fluently than in TypeScript, because the type system distills the feedback signal in a way weakly-typed languages don't. The conditions push AI toward continuously improving capability in exactly these languages, while in weakly-typed languages it has to learn from noisy reward signals indefinitely, or learn to infer the relationships between data structures that a more advanced type system might have made explicit.

Third, in dependently-typed languages, spec and proof are separable artifacts. This separation makes the distinction between the "what" and the "how" explicit, allowing you to ignore the "how" until you actually need to know. Embracing this means that the entire process of designing how the software works becomes a conversation about the "what", while the "how" is left as an exercise to the agent.

You might try and make the claim that verification technologies just move the problem from buggy code to incomplete or underdeveloped specs, but even this would be an improvement because of the *queryability* it unlocks. The separation of spec and

proof means we can search the spec for questions about "what" and only embark on the more tedious search of the proof on the rarer occasion that we genuinely need to know how our

A formal specification is one you can ask precise questions of and get rigorous answers back. The price of inference on any question of a system's behavior drops dramatically when it has one.

The skeptic's counter is that bigger AI models with more data and training compute will get arbitrarily good at JavaScript and the language won't matter. The honest answer is yes. The capability will continue to climb in every language. But the honest answer is also that the *slope* of that improvement differs quite substantially. Each language presents a reward signal of different cleanliness, and the AI that climbs the JavaScript curve climbs slower than the AI that climbs the Lean curve, because in JavaScript half the work is rediscovering the constraints the type system could have given for free.

A corollary of Greenspun's Tenth Rule applies here:

> Any sufficiently advanced reward harness for weakly-typed code is an ad-hoc, informally-specified, bug-ridden, slow implementation of half a dependent type system.

The two trajectories don't converge as we scale AI capability. Instead, the gap widens, because the language's amenability to static analysis determines the slope.

While effort and attention are resources that have to be split between performance and correctness, the idea that powerful high-level languages (the ones best suited for verification) can't meet the performance characteristics of popular industry choices is wrong.

There is a long-running debate about whether the Sufficiently Smart Compiler exists.

In principle, compilers of high-level languages can produce code that is more runtime-efficient than hand-tuned low-level code because the compiler is free to decide the strategy that will accomplish the goal most efficiently, and high-level languages excel at separating goals from strategies. In practice, high-level languages remain slower than their low-level counterparts in a number of important situations.

The argument for SSCs is sound. It is *true* that an arbitrarily smart compiler will translate increasingly high-level descriptions into ever more efficient implementations. The contention is that such a compiler does not exist in the real world and so if we care about performance, we must opt for hand-tuned low-level code.

But the cost of intelligence is plummeting.

Similar to the CapEx ↔ OpEx tradeoff for training engineers,

code. Like all tradeoffs of this kind, there is some crossover point in time where the cumulative cost reduction afforded by that capital investment exceeds its cost, which causes preferences to flip. We're on the cusp of that crossover point now.

Interaction-net runtimes are closing the gap between the languages you can reason about and the languages that run fast. Pure functional programs run *directly* on the silicon with no extraction to a different model of computing. The program you reason about is the program that executes. The performance tax that justified the historical bet has compressed: on workloads with available parallelism, interaction-net evaluation can match or beat imperative execution outright. The runtime story isn't an AI story — it's a separate development that happens to land at the same time, and the two complement each other.

The historical bet that powerful languages would never scale because their adoption cost was prohibitive assumed fluency was confined to humans. AI re-prices this fluency as a one-time investment that doesn't scale with volume.

The conditions that kept verified-software languages niche are dissolving.

I've spent years writing Haskell professionally. I love the language and have tried, repeatedly, to convince teams to use it.

The arguments for Haskell are well-known: clear, concise code that usually works the way you expect on the first try.

The arguments against it are well-known too: hiring availability, training difficulty, performance predictability — they're just increasingly irrelevant in a world where AI does code generation.

The comparative scarcity of engineers who can wield Haskell and similar tools fluently made it a risky choice; it would cause longer vacancy rates in any job, and taking on the responsibility of training those engineers was too expensive to justify. Every engineering org I have worked for gives the same argument against using it: it exposes the business to hiring risks without an obviously defensible benefit to the bottom line.

I don't really care about Haskell specifically. What I want are the *benefits* Haskell offers: clear, concise descriptions of how code will behave, types that make implementations honest about themselves, the elimination of vast classes of bugs at compile

time. Haskell is one delivery mechanism for those benefits. AI agents writing in even more powerful languages — Lean, F*, Rocq — are a different and dramatically better one.

the costs we've come to expect, then those costs were only contingent on historical circumstance, not an inviolable law of computing.

Haskell didn't get outcompeted because it's too rigorous. It got outcompeted because the rigor it enables is *too expensive for humans alone to deliver*.

Where is this Going?

The economics of software development are shifting rapidly. Not every project that could be verified will be. That's not the claim. The claim is that *AI lowers the importance threshold a project must clear before verification becomes the right answer*, and that threshold is falling.

That downward movement accelerates via a few mechanisms. As more projects cross the threshold, the corpus of verified software grows. Verified-software training data is qualitatively richer per token than weakly-typed code: every line carries the conditional structure the type system imposed, the proof obligations annotate intent more densely than tests do, and the program-and-proof together teach a model both what works *and*

why. Intelligence gains per token are higher in this regime. Better models lower the threshold further, and more projects cross it. This is the flywheel.

about this is that most production software has fuzzy specs. UI flows, ML pipelines, or distributed systems running under adversarial conditions.

Formal methods have always struggled with fuzzy specs, and that doesn't change. What changes is what the new substrate buys you. Before AI, using a dependently-typed language taxed every line you wrote regardless of how much you formally specified. Now that AI absorbs the writing tax, a dependently-typed scaffold can carry verified hot paths surrounded by potentially unverified, but queryable, specification code. It also does so without paying the writing cost on the unverified parts.

The formal substrate bounds drift between AI's plan and the resulting code even when no proofs are written. Verification stops being all-or-nothing and becomes surgical.

This type of technology isn't going to magically make everyone specify every relevant property of their programs. But when you start with this scaffold, you gain the ability to incrementally ask the system how it behaves. It can tell you whether those properties have been proven. If they haven't, it can write the

proposition and attempt the proof. And if it can't, at least you'll know.

Some are becoming more obvious by the day. The role of the senior engineer is already changing from "person who writes the code" to "person who can specify what is supposed to be true and read enough of the implementation to co-sign it." This is simply the logical conclusion of that story. The competitive advantage for software professionals is shifting from *velocity-of-implementation* to *precision-of-specification*. The technologies underpinning the languages we currently dismiss as academic will increasingly become production tools we depend on.

I think this trajectory is good. I think it is what software was always supposed to look like. We were just stuck waiting for the cost structure to change… and now it is.

What Work Needs to Get Done?

If this plays out, three pieces of infrastructure have to mature simultaneously.

First, verifiable software has to be fast to run. The runtime performance ceiling has to keep rising — interaction nets, quantitative type theory, optimal reduction, whatever else compiles to bare metal without paying abstraction taxes.

Without this, the verify *and* perform outcome from earlier collapses back into a verify *or* perform tradeoff.

AlphaProof, LeanDojo, and ReProver has to become routine and reliable, not a research curiosity. Models that can fill in proofs on demand are what make the writability-stops-mattering claim concrete rather than aspirational. The worry that AI-generated proofs might be wrong is the wrong worry: writing a proof is creative work, but checking one is mechanical, and we've understood the rules for centuries. Lean's kernel is roughly five thousand lines and isn't AI. By the Curry-Howard Isomorphism, the asymmetry between proof and verification is the same as between writing a program and running it.

Third, machine-assisted specification has to become ergonomic. Specifications are the scarce artifact in this future. The bottleneck moves from "can a model produce code?" (mostly solved) to "can a model help a human author a faithful formal specification — proposing properties, surfacing implicit assumptions, flagging where the spec under-determines the system?" This is the part of the workflow where human judgment is irreducible and machine assistance is leverage, not replacement. The specification is the place authorship has to stay.

If all of these converge, we'll have a new way to deliver trustworthy software in situations where safety is desirable but where the budgets don't permit it.

implementations, queryable end-to-end.

We spent the last three decades putting software in charge of everything. Now we're putting AI in charge of software. The stakes have never been higher.

If you're building something where you're betting your life or livelihood on whether your software does the right thing — and you have a suspicion that test coverage isn't going to let you sleep easy — I want to hear from you.

Formal Vibefication is how you avoid ruin.

New research posts, delivered by email: subscribe.
